

APSQ: Additive Partial Sum Quantization with Algorithm-Hardware Co-Design

Yonghao Tan*, Pingcheng Dong*, Yongkun Wu, Yu Liu, Xuejiao Liu, Peng Luo,
Shih-Yang Liu, Xijie Huang, Dong Zhang, Luhong Liang, Kwang-Ting Cheng[†]

The Hong Kong University of Science and Technology, AI Chip Center for Emerging Smart Systems (ACCESS)

Abstract—DNN accelerators, significantly advanced by model compression and specialized dataflow techniques, have marked considerable progress. However, the frequent access of high-precision partial sums (PSUMs) leads to excessive memory demands in architectures utilizing input/weight stationary dataflows. Traditional compression strategies have typically overlooked PSUM quantization, which may account for 69% of power consumption. This study introduces a novel Additive Partial Sum Quantization (APSQ) method, seamlessly integrating PSUM accumulation into the quantization framework. A grouping strategy that combines APSQ with PSUM quantization enhanced by a reconfigurable architecture is further proposed. The APSQ performs nearly lossless on NLP and CV tasks across BERT, Segformer, and EfficientViT models while compressing PSUMs to INT8. This leads to a notable reduction in energy costs by 28-87%. Extended experiments on LLaMA2-7B demonstrate the potential of APSQ for large language models. Code is available at <https://github.com/Yonghao-Tan/APSQ>.

Index Terms—Hardware-aware quantization, partial sum quantization, dataflow, DNN, Transformer

I. INTRODUCTION

Recent advancements in deep neural networks (DNNs) have driven substantial progress in computer vision (CV) and natural language processing (NLP) tasks, including image classification [1], semantic segmentation [2], [3], object detection [4], and question answering [5]. Furthermore, the rise of generative large language models (LLMs) [6] has profoundly influenced various applications. Central to this advancement is the Transformer architecture [7], renowned for its self-attention mechanism that effectively captures long-range dependencies. However, Transformers are notorious for their intensive computation and memory requirements. For instance, semantic segmentation for self-driving vehicles involves processing over 20,000 tokens, while BERT models must handle tokens with substantial hidden dimensions, such as 4,096, resulting in considerable memory overhead. These challenges significantly impact hardware deployment and acceleration. To address these obstacles, the algorithm community has developed model compression techniques, including quantization [8]–[11], pruning [12], and distillation [13].

To further accelerate DNNs at the hardware level, prior works have focused on designing dedicated accelerators utilizing diverse dataflows [14]–[18]. They can be categorized into Input Stationary (IS), Weight Stationary (WS), and Output

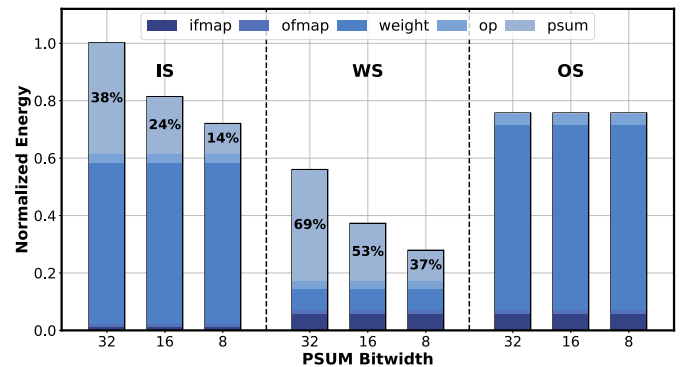


Fig. 1: Energy breakdown of IS, WS, and OS dataflows for the BERT-Base model with 128 input token-length.

Stationary (OS). The IS reduces data movement by keeping input tiles stationary [15] to achieve minimum access to the input buffer. The WS tries to minimize weight movement from external DRAM due to insufficient on-chip weight buffer [16]. The OS achieves optimization by reusing partial sum (PSUM) in the output registers [14], enhancing computational efficiency in scenarios with extensive accesses of PSUMs. In addition, Chen et al. [15] proposed Row Stationary (RS) dataflow, which aims to minimize the energy consumption of data movement for convolutional neural networks (CNNs). However, since Transformers rarely employ standard convolution, the RS dataflow has not been widely adopted. The effectiveness of these dataflows is contingent upon several critical aspects, including the layer configuration, degree of parallelism, and on-chip SRAM size. Addressing this, Tu et al. [16] proposes a reconfigurable architecture specifically designed to leverage the unique characteristics of these diverse dataflows optimally. Besides, recent Transformer accelerators [17], [18] employ Score/Key stationary dataflows to accelerate the attention, which is aligned with the dataflows introduced above.

The key distinction among the aforementioned dataflows lies in the fact that the IS and WS frequently store and fetch PSUMs from SRAM/DRAM for accumulation, while OS updates PSUMs directly within low-cost registers. We perform an energy consumption analysis following the methods adopted in [16] as shown in Fig. 1. It can be seen that PSUMs constitute a large portion of the energy for IS and WS compared with input feature map (ifmap), output feature map (ofmap), weight, and multiply-add operations (op), especially with larger PSUM bit-width, up to 69% of

*Both authors contributed equally to this research.

[†]Corresponding author.

total power consumption. Since PSUMs are obtained by matrix accumulation, they need to be stored in higher precision (e.g., INT16/32) than the quantized weights/activation that are stored in INT8 or lower. Recent studies [19], [20] have proposed PSUM quantization (PSQ) technique to reduce analog-to-digital converter (ADC) overheads of high-precision PSUMs in ReRAM-based accelerator. Nevertheless, the PSUMs after ADC are dequantized to their original bit-width and stored in on-chip SRAM, where the memory access overhead of PSUMs fails to be alleviated.

In this study, we introduce a novel additive PSUM quantization strategy denoted as APSQ, which, to the best of our knowledge, is the first to address challenges in IS and WS encountered with large PSUMs. The APSQ enables each quantizer to consider the cumulative prior outcomes rather than just focusing on the current PSUM. In addition, we propose a grouping strategy to improve accuracy with a reconfigurable architecture to support various grouping configurations. The main contributions of this paper are summarized as follows:

- We reveal the critical role of high-precision PSUM access and storage in both IS and WS dataflows, refining an analytical framework with PSUM precision awareness.
- A novel method termed APSQ is designed by introducing the accumulation effect into the quantizer, thereby achieving INT8 PSUM storage and accesses.
- We propose a grouping strategy integrating PSUM quantization and APSQ within a reconfigurable architecture, efficiently accommodating various group sizes with minimal additional hardware resources.
- The INT8 APSQ minimizes accuracy loss to 0.16% for BERT-Base, 0.61% and 0.83% for Segformer-B0 and EfficientViT-B1, while the energy costs are saved by 28-87% for the IS and WS. We further apply APSQ for LLaMA2-7B and achieved only 0.59% accuracy drop while providing up to $31.7\times$ energy savings, highlighting its significant potential for LLMs.

II. PRELIMINARY

A. Energy Efficiency Analysis

In the realm of DNN accelerators, a key factor is energy efficiency in terms of operations/watt, significantly influenced by precision, layer size, dataflow, and architecture. This study adopts the methodologies from [15], [16] to examine energy efficiency in a typical DNN accelerator system shown in Fig. 2. This system includes a multiply-accumulate (MAC) array, on-chip SRAM, off-chip DRAM, and a top controller. The MAC array is organized based on P_o , P_{ci} , and P_{co} , which define parallelism in the output feature map, input channels, and output channels, respectively. The on-chip SRAM comprises a 128KB input/output buffer and a 64KB weight buffer, while the off-chip DRAM capacity is assumed sufficient, and the top controller manages module configuration. Following [16], the total energy cost E_{total} is:

$$E_{total} = N_d \cdot E_{dram} + N_s \cdot E_{sram} + N_m \cdot E_{mac} \quad (1)$$

where the E_{dram} , E_{sram} , and E_{mac} represent the energy cost for each access to DRAM and SRAM, as well as for a single MAC operation, respectively. The N_d , N_s , and N_m correspond to the total number of accesses to DRAM and SRAM, and the overall count of MAC operations utilized, with energy cost values taken from [21]. Since the number of MACs for a specific DNN is constant, the energy efficiency largely depends on $E_{dram/sram}$ as proved by [16].

In the analytical framework proposed by [15], [16], weights, activations, and partial sums (PSUMs) are all stored in INT16 format. However, in certain integer-only quantized DNN accelerators, PSUMs may require higher precision, such as INT32, to maintain accuracy [22], [23]. Consider a neural network where both weights and activations are quantized to INT8 (W8A8). During a multiply-accumulate (MAC) operation, the product of the quantized weight and activation results in an INT16 value. When these products are accumulated along the input channel dimension, denoted as C_i , the accumulation depth plays a crucial role in determining the required bit width to prevent overflow. Specifically, to ensure numerical stability, the PSUM should be stored using $16 + \log_2(C_i)$ bits. For instance, in the Feed-Forward Network (FFN) of BERT-Large [5], where $C_i = 4096$ in the MLP layer, this results in a required PSUM precision of up to 28 bits to accommodate the full accumulation range without overflow. Given that memory hierarchy designs are typically byte-based, PSUM is generally stored with 32-bit precision.

To accommodate this variation, we revise the framework by introducing a precision factor as below:

$$N_{d/s} = S_i \cdot N_{d/s}^i + S_w \cdot N_{d/s}^w + \beta \cdot S_o \cdot N_{d/s}^p + S_o \cdot N_{d/s}^o \quad (2)$$

the total number of access to DRAM/SRAM for ifmap, weight, PSUM, and ofmap is defined as $N_{d/s}^i$, $N_{d/s}^w$, $N_{d/s}^p$ and $N_{d/s}^o$, with S_i , S_w and S_o indicating the size of ifmap, weights, and ofmap. The β is the ratio of PSUM precision to that of feature map/weight. For instance, β would be 4 if PSUM is in INT32 for an INT8 DNN. In this work, we merely focus on INT8 DNN with WS or IS, since the OS does not struggle with the precision of PSUM.

1) *Input Stationary*: The IS-based accelerator keeps the input tiles stationary within the MAC array's registers, facilitating their reuse by weights to update the PSUMs in the output buffer. Similarly, the $N_s^{i/w/p/o}$ is summarized in equation (3):

$$N_s^w = \begin{cases} 1 + \left\lceil \frac{H_i}{P_{ih}} \right\rceil \left\lceil \frac{W_i}{P_{iw}} \right\rceil, & S_w < B_w \\ 2 \left\lceil \frac{H_i}{P_{ih}} \right\rceil \left\lceil \frac{W_i}{P_{iw}} \right\rceil, & S_w \geq B_w \end{cases}, N_s^i = 2 \quad (3)$$

$$N_s^p = \begin{cases} 2 \left(\left\lceil \frac{C_i}{P_{ci}} \right\rceil - 1 \right), & \frac{C_o}{P_{co}} \cdot \tilde{S}_p < B_o \\ 4 \left(\left\lceil \frac{C_i}{P_{ci}} \right\rceil - 1 \right), & \frac{C_o}{P_{co}} \cdot \tilde{S}_p \geq B_o \end{cases}, N_s^o = 2$$

the weight buffer size is denoted by B_w , and the height and width of the enlarged ifmap are H_i and W_i , respectively. Besides, the ifmap tile is shaped by P_{ih} and P_{iw} following $P_i = P_{ih} \times P_{iw}$. Notably, the PSUM of IS is smaller than that of WS since the ifmap tile reused by weights is

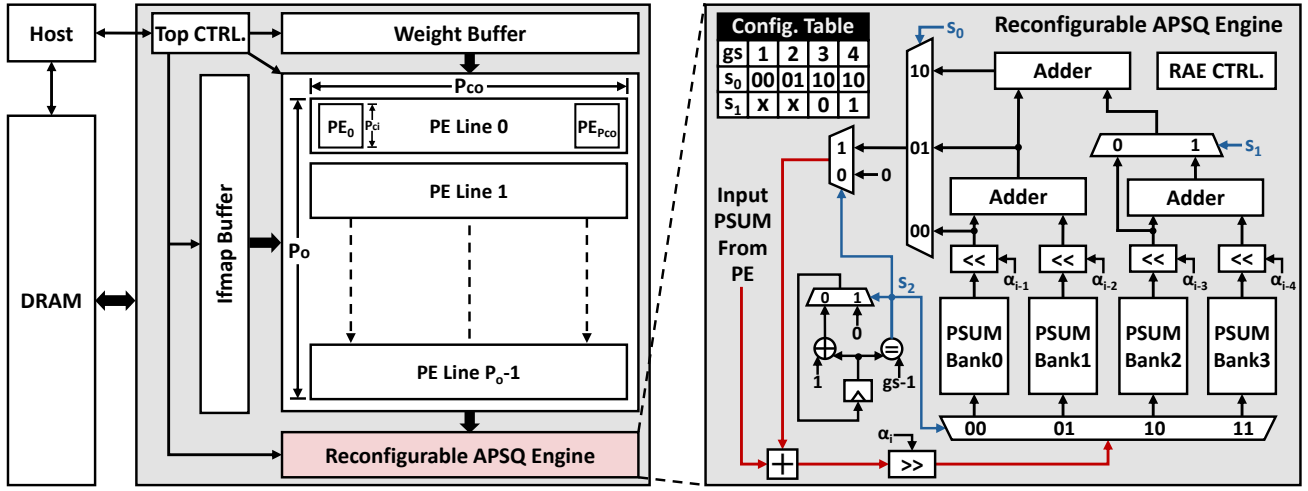


Fig. 2: The analytical DNN accelerator and the Reconfigurable APSQ Engine (RAE) in this work.

generally small. Consequently, we scale the $\tilde{S}_p$ by C_o/P_{co} . The $N_p^{i/w/p/o}$ for DRAM is derived as:

$$N_d^w = \begin{cases} 1, & S_w < B_w \\ \left\lceil \frac{H_i}{P_{ci}} \right\rceil \left\lceil \frac{W_i}{P_{iw}} \right\rceil, & S_w \geq B_w \end{cases}, N_d^i = 1$$

$$N_d^p = \begin{cases} 0, & \frac{C_o}{P_{co}} \cdot \tilde{S}_p < B_o \\ 2 \left(\left\lceil \frac{C_i}{P_{ci}} \right\rceil - 1 \right), & \frac{C_o}{P_{co}} \cdot \tilde{S}_p \geq B_o \end{cases}, N_s^o = 1$$

2) *Weight Stationary*: In contrast, the WS-based accelerator loads ifmap tiles to update the PSUMs via reusing $P_{ci} \times P_{co}$ kernel weights, and the ofmap tile is generated after finishing PSUM accumulation. According to this, the components of N_s can be derived as:

$$N_s^i = \begin{cases} 1 + \left\lceil \frac{C_o}{P_{co}} \right\rceil, & \tilde{S}_i < B_i \\ 2 \left\lceil \frac{C_o}{P_{co}} \right\rceil, & \tilde{S}_i \geq B_i \end{cases}, N_s^w = 2$$

$$N_s^p = \begin{cases} 2 \left(\left\lceil \frac{C_i}{P_{ci}} \right\rceil - 1 \right), & \frac{H_o \cdot W_o}{P_o} \cdot \tilde{S}_p < B_o \\ 4 \left(\left\lceil \frac{C_i}{P_{ci}} \right\rceil - 1 \right), & \frac{H_o \cdot W_o}{P_o} \cdot \tilde{S}_p \geq B_o \end{cases}, N_s^o = 2$$

where C_i and C_o represent the dimensions of input and output channels, the B_i and B_o indicate the capacities of ifmap and ofmap in bytes. The ofmap's height and width are denoted as H_o and W_o . Besides, the input tile size $\tilde{S}_i$ is enlarged based on output tiles, kernels, and strides, as explained in [16]. The $\tilde{S}_p$ is the size of tiled PSUM calculated as $\beta \times P_o \times P_{co}$, where P_{oh} and P_{ow} are the height and width of the output/PSUM tile, constrained by $P_o = P_{oh} \times P_{ow}$. If exceeding the B_i or B_o , extra read to both the SRAM and DRAM occur, the $N_d^{i/w/p/o}$ is defined as:

$$N_d^i = \begin{cases} 1, & \tilde{S}_i < B_i \\ \left\lceil \frac{C_o}{P_{co}} \right\rceil, & \tilde{S}_i \geq B_i \end{cases}, N_d^w = 1$$

$$N_d^p = \begin{cases} 0, & \frac{H_o \cdot W_o}{P_o} \cdot \tilde{S}_p < B_o \\ 2 \left(\left\lceil \frac{C_i}{P_{ci}} \right\rceil - 1 \right), & \frac{H_o \cdot W_o}{P_o} \cdot \tilde{S}_p \geq B_o \end{cases}, N_d^o = 1$$

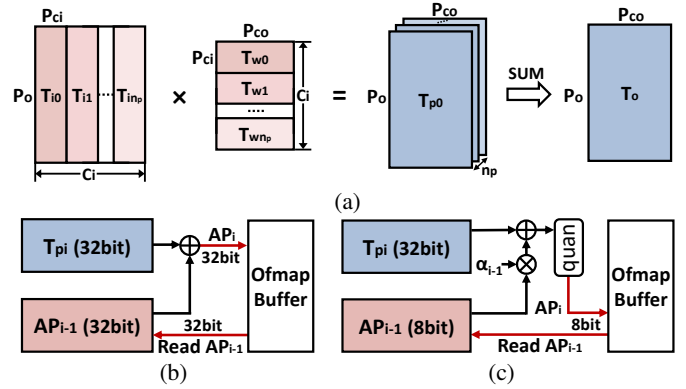


Fig. 3: (a) An example of pointwise convolution with tile-based computation scheme, (b) typical partial sum accumulation, (c) proposed INT8 additive partial sum quantization ($gs = 1$).

B. Quantization

Quantization has emerged as an effective strategy to alleviate computational and memory overheads, particularly advantageous for deployments on hardware accelerators, and can be expressed as:

$$\tilde{x} = \alpha \cdot Q_k(x) = \alpha \cdot \left[\text{Clip} \left(\frac{x}{\alpha}, Q_n, Q_p \right) \right] \quad (7)$$

where $Q_k(\cdot)$ represents the k -bit quantization function that compresses the high-precision input data to low-bit INT- k with a scaling factor α . Specifically, the $Q_k(x)$ constrains $\frac{x}{\alpha}$ within the range $[-2^{k-1}, 2^{k-1} - 1]$ or $[0, 2^k - 1]$ for the signed and unsigned quantization, bounded by Q_n and Q_p . The dequantized $\tilde{x}$ can be retrieved through re-scaling $Q_k(x)$ by α , which is determined either using the min-max technique [9] or the learnable alternative [10]. In this work, we choose the learnable method LSQ [10] to quantize both weights and activations/PSUMs, among which the scaling factor α of PSUMs are forced to power-of-two format by learning $2^{\lfloor \log_2^{\alpha} \rfloor}$ via straight through estimator (STE) [24], [25]. In this way, the multiplication in the re-scaling could be replaced by the hardware-efficient shift operation.

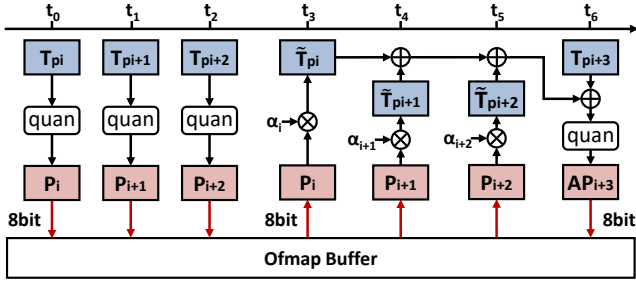


Fig. 4: Workflow of additive partial sum quantization with $gs = 3$.

III. METHOD

A. Additive Partial Sum Quantization

As highlighted in Section II-A, tile-based computation (TBC) is a primary method in DNN accelerators due to computational limitations. This technique frequently involves additive interactions with PSUMs to produce the final output tile. Defining T_i , T_w , T_p , and T_o as the tiles for ifmap, weight, PSUM, and ofmap severally, a typical PSUM accumulation example with TBC is illustrated in Fig. 3b. The ofmap tile T_o is derived from the accumulation of multiple PSUMs. Thus, the TBC could be formulated as follows:

$$T_o = \sum_{i=0}^{n_p-1} T_{pi}, \quad n_p = \left\lceil \frac{C_i}{P_{ci}} \right\rceil \quad (8)$$

where n_p signifies the number of PSUM tile T_{pi} for $0 \leq i \leq n_p - 1$. Although TBC ensures computational efficiency, the necessity for high precision (INT32) in PSUMs can still introduce heavy energy overheads, as the numeric results of PSUM accumulation are generally large in Transformers, which is proved in Section IV. Besides, the typical PSUM accumulation could be formulated as follows:

$$AP_j = \sum_{i=0}^j T_{pi}, \quad T_o = AP_j + \sum_{i=j+1}^{n_p-1} T_{pi} \quad (9)$$

where each AP_j represents the additive PSUM after the j^{th} PSUM tile T_{pj} . Since the numeric results of PSUM accumulation are generally large in Transformers, they are required to be stored in INT32, which is proved in Section II-A. To achieve low-bit memory accesses across all AP_s , we propose an additive PSUM quantization approach termed APSQ, detailed as follows:

$$AP_i = Q_k^i(T_{pi} + \alpha_{i-1} \cdot AP_{i-1}), \quad \text{for } 1 \leq i < n_p. \quad (10)$$

In the APSQ, each AP_i is recursively determined by applying Q_k^i to the sum of T_{pi} and the preceding additive PSUM AP_{i-1} which is depicted in Fig. 3c. The initial additive PSUM AP_0 is set as $Q_k^0(T_{p0})$. The output tile T_o is derived upon completing the quantization of the final PSUM. Consequently, T_o can be succinctly expressed as AP_{n_p-1} . In this way, each AP_s could be accessed by low-bit precision while the quantizer considers both the current high-precision PSUM and its previous AP_s .

Algorithm 1 Grouping Strategy

Input: Total number of PSUM tiles n_p , PSUM tile set $T_p = [T_{p0}, \dots, T_{pn_p-1}]$, scaling factor set $\alpha = [\alpha_0, \alpha_1, \dots, \alpha_{n_p-1}]$ and the group size gs .

Output: Output tile T_o .

- 1: α is stored in the register list, $n_p > 0$ and $AP_{i<0}^* = 0$
- 2: $T_o = \alpha_0 \times Q_k^0(T_{p0})$ ▷ Initialize T_o
- 3: **for** $i = 0$ to $n_p - 1$ with step gs **do**
- 4: Read gs previous PSUMs from buffer
- 5: $AP_{i-1}^* = \sum_{i=i-gs}^{i-1} AP_i^* \times \alpha_i$ ▷ Accumulation
- 6: $AP_i^* = Q_k^i(AP_{i-1}^* + T_{pi})$ ▷ Perform APSQ
- 7: Write AP_i^* to buffer
- 8: **for** $j = i + 1$ to $\min(i + gs - 1, n_p - 1)$ **do**
- 9: **if** $j < n_p - 1$ **then**
- 10: $AP_j^* = Q_k^j(T_{pj})$ ▷ Perform PSUM quantization
- 11: Write AP_j^* to buffer
- 12: **else** ▷ Final output tile T_o
- 13: Read $n_p - i + 1$ previous PSUMs from buffer
- 14: $T_o = \alpha_{n_p-1} \times Q_k^{n_p-1}(\sum_{l=i}^{n_p-2} AP_l^* \times \alpha_l + T_{pn_p-1})$
- 15: **end if**
- 16: **end for**
- 17: **end for**

B. Grouping Strategy

However, the accuracy may suffer since each PSUM interacts with multiple quantizers, leading to heavy approximation errors due to repeated rounding during quantization in APSQ. To mitigate this issue, reducing the quantization frequency is essential, as it can minimize the additional rounding error. Building on this insight, we proposed a grouping strategy that combines APSQ with typical quantization detailed in Algorithm 1. Specifically, the PSUM tile set is partitioned into groups of size gs , where APSQ is applied only once per group, targeting the accumulation of the current input PSUM tile and the previous gs dequantized PSUM tiles. An workflow example of the grouping strategy is depicted in Fig. 4. Each group of gs quantized PSUMs is stored in the Ofmap Buffer using INT8 precision per PSUM and is sequentially read from the buffer for accumulation, with dequantization performed at INT8 precision per PSUM read. Notably, the grouping strategy maintains the same total memory read and write operations for APSQ with both $gs = 1$ and $gs > 1$.

C. Reconfigurable APSQ Architecture

While the grouping strategy offers opportunities to enhance accuracy, an excessively large gs may introduce additional memory overhead. Moreover, different models exhibit varying optimal values for gs , as demonstrated in Table I. To support APSQ with various gs , we designed a Reconfigurable APSQ Engine (RAE) as depicted in Fig. 2. The RAE features a PSUM Buffer with four SRAM banks, shifter-based quantization/dequantization modules, adders, and a dedicated RAE controller. RAE's work mode is governed by static encodings s_0 and s_1 , in conjunction with a dynamic encoding s_2 . Static

TABLE I: Accuracy comparison between Baseline and APSQ methods across various models and tasks.

Model	Task	Baseline	gs=1	gs=2	gs=3	gs=4
BERT-Base	QNLI	91.32	90.26	90.77	91.12	91.03
	MNLI	84.08	82.27	83.12	83.43	83.54
	RTE	74.73	74.01	74.01	73.29	75.81
	STS-B	87.89	86.94	87.31	87.60	87.61
	MRPC	87.99	87.25	87.01	87.75	87.01
	CoLA	53.40	50.84	51.27	52.59	52.36
Segformer-B0	ADE20K	36.72	35.83	36.11	35.97	35.85
EfficientViT-B1		39.48	37.45	38.65	38.41	38.47

encodings s_0 and s_1 are derived from a predefined table based on the group size gs , configuring the RAE multiplexers to select appropriate PSUM tiles P_i for different gs . The dynamic encoding s_2 determines whether an APSQ or a PSUM quantization operation is performed.

For instance, when $gs = 1$, s_0 is set to 00, s_1 is irrelevant, and s_2 remains 1. In this mode, each PSUM P_i generated by the PE array prompts the RAE to retrieve the previous INT8 PSUM P_{i-1} from PSUM Bank0, dequantize and accumulate it with P_i . The result is quantized via a shift operation and written back to Bank0. When $gs = 4$, the configuration adjusts: s_0 becomes 10, s_1 is set to 1, and s_2 dynamically toggles. With s_2 set to 0, incoming PSUMs are quantized and stored in the corresponding PSUM Banks without APSQ. After four rounds of PSUM quantization, s_2 switches to 1, triggering the simultaneous retrieval of four PSUMs, P_{i-4} to P_{i-1} , from all four PSUM Banks. These are processed through a two-stage adder pipeline, accumulated with P_i , quantized, and then stored in PSUM Bank3. This flexible architecture allows the RAE to efficiently accommodate various group sizes, effectively replacing conventional PSUM accumulation and storage methods in IS/WS-based DNN accelerators.

IV. EXPERIMENTAL RESULTS

A. Experiment Setup

In this section, we evaluate the model accuracy and hardware performance of the proposed APSQ method across both NLP and CV domains. For the NLP experiments, we adopt the BERT-Base model with an input token length of 128 [5] and benchmark it using the GLUE [26] dataset, which includes tasks such as QNLI, MNLI, RTE, STS-B, MRPC, and CoLA. In the CV domain, we focus on semantic segmentation using the ADE20K [27] dataset, comprising over 20,000 images with 150 distinct classes. We experiment with two models at a 512×512 input resolution: Segformer-B0 [3], which employs a Transformer architecture with vanilla attention, and EfficientViT-B1 [2], a lightweight model that integrates convolution with linear attention. All baseline models in our experiments are quantized to W8A8, and APSQ is incorporated into the quantization-aware training (QAT) process, guided by a full-precision teacher model for knowledge distillation. We employ task-specific metrics from the GLUE benchmark for evaluating NLP tasks and the standard mean intersection over union (mIoU) metric to assess segmentation performance.

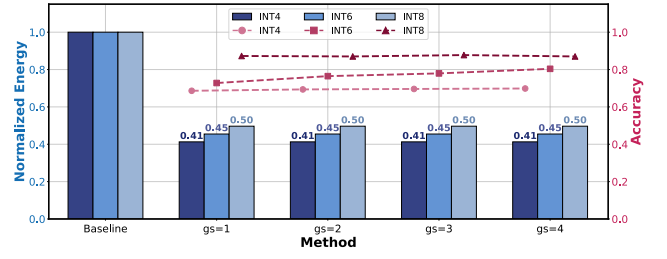


Fig. 5: Normalized energy and accuracy across varied gs settings for MRPC under WS dataflow on BERT-Base.

Furthermore, we apply the proposed techniques to a large language model LLaMA2-7B for zero-shot commonsense reasoning evaluation.

The configuration of the analytical DNN accelerator, as shown in Fig. 2, begins with the allocation of memory buffers: 256KB each for the input feature map (ifmap) and output feature map (ofmap), and 128KB for the weight buffer, while the off-chip DRAM is DDR3. In the MAC array setup, the parallelisms are organized as $P_o = 16$, $P_{ci} = 8$, and $P_{co} = 8$ for BERT-Base, Segformer-B0, and EfficientViT-B1. For large language model (LLM) evaluation, the parallelism settings are adjusted to $P_o = 1$, $P_{ci} = 32$, and $P_{co} = 32$, since the input to the LLM at the decoding stage is a vector, enabling P_o to be set to 1.

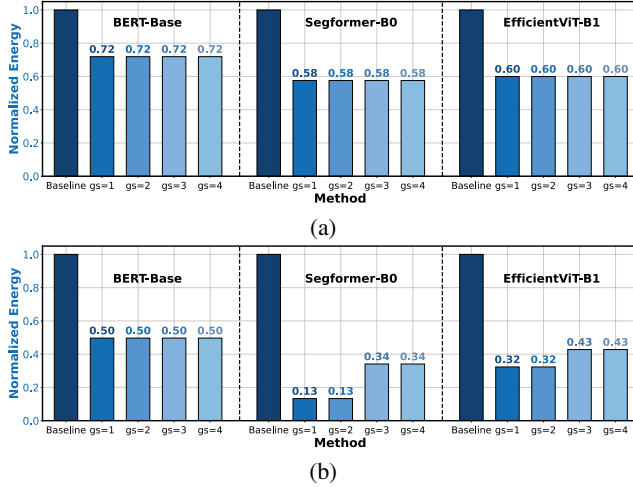
B. Accuracy Analysis

As discussed in Sections III-A and III-B, applying APSQ to all PSUMs (equivalent to $gs = 1$ grouping) significantly degrades performance due to excessive quantization on each PSUM. To validate this, we evaluated accuracy across different gs values in Table I under full INT8 quantization. Results show that $gs = 1$ causes notable accuracy drops, indicating APSQ's insufficiency alone. Although increasing gs generally helps restore accuracy, the improvements are not strictly monotonic. The performance variation across different models and tasks implies that the intrinsic characteristics of each model affect the effectiveness of APSQ, leading to diverse outcomes for different gs values. These observations underscore the necessity of the RAE, which is designed to adapt to varying gs requirements and optimize accuracy accordingly.

Additionally, as shown in Fig. 5, lower PSUM precision consistently reduces energy consumption. However, energy savings weaken below INT8 precision, indicating that reducing PSUM precision further yields minimal benefits and incurs substantial accuracy loss. Hence, adopting INT8 precision for APSQ is technically optimal. Compared to the BERT-Base baseline, the average accuracy metrics determined from the best results of each individual task decrease by only 0.16%, demonstrating a minimal trade-off. For the semantic segmentation task using Segformer-B0 and EfficientViT-B1, the mIoU decreases by just 0.61% and 0.83% with $gs = 2$, indicating the INT8 setup with a suitable task-specific gs could strike a balance between accuracy and energy savings.

TABLE II: Hardware synthesis resource consumptions

	Area (μm^2)
Baseline DNN Accelerator	1873408
RAE	86410
DNN Accelerator w/ RAE	1933674

Fig. 6: Normalized energy across varied gs settings, and models under (a) IS dataflow, (b) WS dataflow.

C. Hardware Performance Analysis

In continuation of the prior analyses on accuracy, we evaluate the energy costs associated with different gs values, following the methods detailed in Section II-A. As depicted in Fig. 6, APSQ achieves normalized energy consumption reductions of 28%, 42%, and 40% for BERT-Base, Segformer-B0, and EfficientViT-B1, respectively, in IS dataflow. The gs parameter has minimal influence on energy savings for IS, given the typically small PSUM tile size. However, its effect becomes more significant in WS dataflow, where large PSUMs must be buffered. For WS, varying gs values lead to a uniform 50% energy reduction in BERT-Base, owing to its short token length. In contrast, the impact varies for semantic segmentation models: Segformer-B0 maintains an 87% energy saving at $gs = 1$ and $gs = 2$, but this benefit declines to 66% at $gs = 3$ and $gs = 4$. Similarly, EfficientViT-B1 achieves a 68% energy reduction for $gs = 1$ and $gs = 2$, decreasing to 57% at higher gs values. This decline occurs as larger PSUM sizes surpass buffer capacity at higher gs , particularly with high-resolution input, resulting in additional DRAM operations.

Consequently, when the PSUM is fully stored on-chip, energy consumption primarily depends on the PSUM precision. However, when the buffer size becomes insufficient, as observed in Segformer-B0 and EfficientViT-B1 under WS, gs significantly impacts energy consumption. This trade-off between accuracy and energy efficiency emphasizes the necessity of a reconfigurable architecture that can dynamically adjust to various gs settings, underscoring the essential role of the RAE. We implement a DNN accelerator with and without RAE using Verilog HDL and obtain the hardware resources via Synopsys Design Compiler at 28-nm technology with a frequency constraint of 250 MHz. The accelerator without

RAE served as the baseline. As presented in Table II, the accelerator with RAE incurred only a 3.21% increase in the synthesized area while achieving a 28-87% reduction in energy consumption as discussed before.

D. Experiment on Large Language Model

TABLE III: Accuracy of Baseline and APSQ across seven Zero-shot Common Sense Reasoning tasks on LLaMA2-7B.

Method	BoolQ	PIQA	HellaS.	WinoG.	Arc-e	Arc-c	OBQA
Baseline	77.80	79.22	76.64	69.69	75.25	47.10	43.40
gs=1	75.26	76.82	72.99	65.75	71.38	42.58	38.60
gs=2	75.93	77.09	74.94	67.48	73.86	46.42	42.00
gs=3	76.45	78.84	75.43	68.43	73.40	47.18	41.80
gs=4	76.82	78.45	76.01	67.96	74.75	47.35	42.80

TABLE IV: Normalized energy across varied gs settings under IS and WS on LLaMA2-7B.

Dataflow	Baseline	gs=1	gs=2	gs=3	gs=4
IS	1.02×	1×	1×	1×	1×
WS	31.7×	1×	1×	8.42×	8.42×

Nowadays, large language models (LLMs) represent a significant advancement in the field of AI. The autoregressive nature of LLMs leads to only one token being inferred at a time during the generation phase. To simulate this scenario while keeping the total number of MAC operations unchanged, we adjust the parallelisms to $P_o = 1$, $P_{ci} = 32$, and $P_{co} = 32$. We implement APSQ within the RoLoRA framework [28] and apply W8A8 quantization for LLaMA2-7B as the baseline. Experiments are conducted on seven Zero-shot Common Sense Reasoning (ZCSR) tasks [29]. As shown in Table III, APSQ results in an average accuracy reduction of only 0.59% when compared to the baseline, evaluated using the best accuracy for each individual task. Despite this small accuracy trade-off, it achieves substantial energy savings of 1.02-31.7× with 4096 sequence length considering both prefilling and decoding stages, as shown in Table IV, demonstrating APSQ's significant potential in LLM applications. The minimal enhancement of APSQ on IS is primarily due to the feature map being a vector, which is considerably smaller than weight. Further explorations will be conducted in our future work.

V. CONCLUSION

This work highlights the significant energy overhead from high-precision PSUM accesses in DNN accelerators, often overlooked in prior research. To address this, we first refine a PSUM-precision-aware analytical framework to evaluate the energy consumption of DNN accelerators. Then, we propose a novel APSQ method enhanced by a grouping strategy and a reconfigurable architecture. Experimental results demonstrate the intricate relationship between group size and PSUM quantization overhead. The INT8 APSQ implementation achieves minimal accuracy loss on the GLUE and ADE20K datasets while significantly reducing energy costs. Further experiments also reveal APSQ's potential for efficient acceleration of LLMs. These findings provide a strategic framework for optimizing DNN accelerators that harness APSQ.

REFERENCES

- [1] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” *arXiv preprint arXiv:2010.11929*, 2020.
- [2] H. Cai, J. Li, M. Hu, C. Gan, and S. Han, “Efficientvit: Lightweight multi-scale attention for high-resolution dense prediction,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, October 2023, pp. 17 302–17 313.
- [3] E. Xie, W. Wang, Z. Yu, A. Anandkumar, J. M. Alvarez, and P. Luo, “Segformer: Simple and efficient design for semantic segmentation with transformers,” *Advances in Neural Information Processing Systems*, vol. 34, pp. 12 077–12 090, 2021.
- [4] Y. Quan, D. Zhang, L. Zhang, and J. Tang, “Centralized feature pyramid for object detection,” *IEEE Transactions on Image Processing*, 2023.
- [5] J. Devlin, “Bert: Pre-training of deep bidirectional transformers for language understanding,” *arXiv preprint arXiv:1810.04805*, 2018.
- [6] H. Touvron, L. Martin, K. Stone, P. Albert, A. Almahairi, Y. Babaei, N. Bashlykov, S. Batra, P. Bhargava, S. Bhosale *et al.*, “Llama 2: Open foundation and fine-tuned chat models,” *arXiv preprint arXiv:2307.09288*, 2023.
- [7] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, “Attention is all you need,” *Advances in neural information processing systems*, vol. 30, 2017.
- [8] B. Jacob, S. Kligys, B. Chen, M. Zhu, M. Tang, A. Howard, H. Adam, and D. Kalenichenko, “Quantization and training of neural networks for efficient integer-arithmetic-only inference,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2018, pp. 2704–2713.
- [9] S. Kim, A. Gholami, Z. Yao, M. W. Mahoney, and K. Keutzer, “I-bert: Integer-only bert quantization,” in *International conference on machine learning*. PMLR, 2021, pp. 5506–5518.
- [10] S. K. Esser, J. L. McKinstry, D. Bablani, R. Appuswamy, and D. S. Modha, “Learned step size quantization,” in *8th International Conference on Learning Representations, ICLR 2020, Addis Ababa, Ethiopia, April 26-30, 2020*, 2020.
- [11] S.-Y. Liu, Z. Liu, and K.-T. Cheng, “Oscillation-free quantization for low-bit vision transformers,” *arXiv preprint arXiv:2302.02210*, 2023.
- [12] Z. Liu, H. Mu, X. Zhang, Z. Guo, X. Yang, K.-T. Cheng, and J. Sun, “Metapruning: Meta learning for automatic neural network channel pruning,” in *Proceedings of the IEEE/CVF international conference on computer vision*, 2019, pp. 3296–3305.
- [13] C. Shu, Y. Liu, J. Gao, Z. Yan, and C. Shen, “Channel-wise knowledge distillation for dense prediction,” in *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 2021, pp. 5311–5320.
- [14] T. Chen, Z. Du, N. Sun, J. Wang, C. Wu, Y. Chen, and O. Temam, “Diannao: A small-footprint high-throughput accelerator for ubiquitous machine-learning,” *ACM SIGARCH Computer Architecture News*, vol. 42, no. 1, pp. 269–284, 2014.
- [15] Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, “Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks,” *IEEE journal of solid-state circuits*, vol. 52, no. 1, pp. 127–138, 2016.
- [16] F. Tu, S. Yin, P. Ouyang, S. Tang, L. Liu, and S. Wei, “Deep convolutional neural network architecture with reconfigurable computation patterns,” *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 25, no. 8, pp. 2220–2233, 2017.
- [17] H. You, Z. Sun, H. Shi, Z. Yu, Y. Zhao, Y. Zhang, C. Li, B. Li, and Y. Lin, “Vitcod: Vision transformer acceleration via dedicated algorithm and accelerator co-design,” in *2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*. IEEE, 2023, pp. 273–286.
- [18] L. Lu, Y. Jin, H. Bi, Z. Luo, P. Li, T. Wang, and Y. Liang, “Sanger: A co-design framework for enabling sparse attention using reconfigurable architecture,” in *MICRO-54: 54th Annual IEEE/ACM International Symposium on Microarchitecture*, 2021, pp. 977–991.
- [19] A. Azamat, F. Asim, J. Kim, and J. Lee, “Partial sum quantization for reducing adc size in rram-based neural network accelerators,” *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, 2023.
- [20] J. Bai, W. Xue, Y. Fan, S. Sun, and W. Kang, “Partial sum quantization for computing-in-memory based neural network accelerator,” *IEEE Transactions on Circuits and Systems II: Express Briefs*, 2023.
- [21] M. Horowitz, “1.1 computing’s energy problem (and what we can do about it),” in *2014 IEEE international solid-state circuits conference digest of technical papers (ISSCC)*. IEEE, 2014, pp. 10–14.
- [22] S. K. Lee, P. N. Whatmough, M. Donato, G. G. Ko, D. Brooks, and G.-Y. Wei, “Smiv: A 16-nm 25-mm² soc for iot with arm cortex-a53, efpga, and coherent accelerators,” *IEEE Journal of Solid-State Circuits*, vol. 57, no. 2, pp. 639–650, 2022.
- [23] S. Ryu, H. Kim, W. Yi, E. Kim, Y. Kim, T. Kim, and J.-J. Kim, “Bitblade: Energy-efficient variable bit-precision hardware accelerator for quantized neural networks,” *IEEE Journal of Solid-State Circuits*, vol. 57, no. 6, pp. 1924–1935, 2022.
- [24] Y. Bengio, N. Léonard, and A. Courville, “Estimating or propagating gradients through stochastic neurons for conditional computation,” *arXiv preprint arXiv:1308.3432*, 2013.
- [25] P. Dong, Y. Tan, D. Zhang, T. Ni, X. Liu, Y. Liu, P. Luo, L. Liang, S.-Y. Liu, X. Huang *et al.*, “Genetic quantization-aware approximation for non-linear operations in transformers,” in *Proceedings of the 61st ACM/IEEE Design Automation Conference*, 2024, pp. 1–6.
- [26] A. Wang, “Glue: A multi-task benchmark and analysis platform for natural language understanding,” *arXiv preprint arXiv:1804.07461*, 2018.
- [27] B. Zhou, H. Zhao, X. Puig, S. Fidler, A. Barriuso, and A. Torralba, “Scene parsing through ade20k dataset,” in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 633–641.
- [28] X. Huang, Z. Liu, S.-Y. Liu, and K.-T. Cheng, “Rolora: Fine-tuning rotated outlier-free llms for effective weight-activation quantization,” *arXiv preprint arXiv:2407.08044*, 2024.
- [29] L. Gao, J. Tow, S. Biderman, S. Black, A. DiPofi, C. Foster, L. Golding, J. Hsu, K. McDonnell, N. Muennighoff *et al.*, “A framework for few-shot language model evaluation,” *Version v0. 0.1. Sept.*, vol. 10, pp. 8–9, 2021.